COMSATS University Islamabad

Attock Campus

Department Of Computer Science

Course	***INFIRMATION SECURITY***
Instructor	***MS AMBREEN GUL***
Lab Assignment no.	***04***

Student Details

Registration No.	***Name***
FA24-BSE-048	***HAROON KHAN***

Task 1: Simple RSA Implementation:

Code:

```python
import random
# Function to find Greatest Common Divisor (GCD)
def gcd(a, b):  2 usages
    while b != 0:
        a, b = b, a % b
    return a
# Function to find Modular Multiplicative Inverse
def multiplicative_inverse(e, phi):  1 usage
    d = 0
    x1, x2 = 0, 1
    y1 = 1
    temp_phi = phi
    # Extended Euclidean Algorithm
    while e > 0:
        temp1 = temp_phi // e
        temp2 = temp_phi - temp1 * e
        temp_phi = e
        e = temp2
        x = x2 - temp1 * x1
        y = d - temp1 * y1
        x2 = x1
        x1 = x
        d = y1
        y1 = y
    if temp_phi == 1:
        return d + phi
# Function to generate RSA public and private keys
def generate_keypair(p, q):  1 usage
    # Step 1: Calculate n
    n = p * q
    phi = (p - 1) * (q - 1)
    # Step 3: Choose random e
    e = random.randrange( start: 2, phi)
    # Check if e and phi are coprime
    g = gcd(e, phi)
    while g != 1:
        e = random.randrange( start: 2, phi)
        g = gcd(e, phi)
    # Step 4: Generate private key d
    d = multiplicative_inverse(e, phi)
    # Public Key = (e, n)
    # Private Key = (d, n)
    return ((e, n), (d, n))
# Function to encrypt message
def encrypt(public_key, plaintext):  1 usage
    # Separate e and n from public key
    e, n = public_key
    # Encrypt each character
    cipher = [(ord(char) ** e) % n for char in plaintext]
```

```python
    return cipher
# Function to decrypt message
def decrypt(private_key, ciphertext):  1 usage
    # Separate d and n from private key
    d, n = private_key
    # Decrypt each number back to character
    plain = [chr((char ** d) % n) for char in ciphertext]
    return ''.join(plain)
# ---------------- MAIN PROGRAM ----------------
# Take prime numbers from user
p = int(input("Enter first prime number: "))
q = int(input("Enter second prime number: "))
# Generate RSA keys
public_key, private_key = generate_keypair(p, q)
# Display keys
print("\nPublic Key:", public_key)
print("Private Key:", private_key)
# Take message input from user
message = input("\nEnter message: ")
# Encrypt the message
encrypted_msg = encrypt(public_key, message)
print("\nEncrypted Message:", encrypted_msg)
# Decrypt the message
decrypted_msg = decrypt(private_key, encrypted_msg)
print("Decrypted Message:", decrypted_msg)
```

Output:

```
"C:\Users\DELL\PycharmProjects\IS assig\.venv\Scripts\python.exe" "C:\Users\DELL\PycharmProjects\IS assig\rsa.py"
Enter first prime number: 41
Enter second prime number: 91

Public Key: (1181, 3731)
Private Key: (2021, 3731)

Enter message: haroon khan

Encrypted Message: [2561, 1133, 3189, 258, 258, 2719, 1549, 2895, 2561, 1133, 2719]
Decrypted Message: haroon khan

Process finished with exit code 0
```

Line-by-Line Explanation of Task 1:

1. Import Module

import random

- *Imports the random module.*
- *Used to generate a random value for e in RSA key generation.*

2. GCD Function

def gcd(a, b):

- *Defines a function named gcd.*

- *Finds the Greatest Common Divisor of two numbers.*

while b != 0:

- *Loop continues until b becomes 0.*

a, b = b, a % b

- *Uses Euclid's Algorithm.*
- *Replaces a with b.*
- *Replaces b with remainder of a ÷ b.*

return a

- *Returns final GCD value.*

3. Multiplicative Inverse Function

def multiplicative_inverse(e, phi):

- *Creates function to calculate modular inverse.*

d = 0

x1, x2 = 0, 1

y1 = 1

temp_phi = phi

- *Initializes variables used in Extended Euclidean Algorithm.*

while e > 0:

- *Loop runs until e becomes 0.*

temp1 = temp_phi // e

*temp2 = temp_phi - temp1 * e*

- *Calculates quotient and remainder.*

temp_phi = e

e = temp2

- *Updates values for next iteration.*

*x = x2 - temp1 * x1*

*y = d - temp1 * y1*

- *Calculates temporary x and y values.*

x2 = x1

x1 = x

- *Updates x values.*

d = y1

y1 = y

- *Updates y values.*

if temp_phi == 1:

return d + phi

- *Returns positive modular inverse value.*

4. Key Generation Function

def generate_keypair(p, q):

- *Defines RSA key generation function.*

*n = p * q*

- *Calculates modulus n.*

*phi = (p - 1) * (q - 1)*

- *Calculates Euler Totient Function.*

e = random.randrange(2, phi)

- *Selects random public exponent.*

g = gcd(e, phi)

- *Finds GCD of e and phi.*

while g != 1:

- *Loop continues until GCD becomes 1.*

e = random.randrange(2, phi)

g = gcd(e, phi)

- *Generates another random value for e.*

d = multiplicative_inverse(e, phi)

- *Calculates private key value d.*

return ((e, n), (d, n))

- *Returns public and private keys.*

5. Encryption Function

def encrypt(public_key, plaintext):

- *Defines encryption function.*

e, n = public_key

- *Separates e and n from public key.*

*cipher = [(ord(char) ** e) % n for char in plaintext]*

- *Converts characters into encrypted numbers.*

return cipher

- *Returns encrypted message.*

6. Decryption Function

def decrypt(private_key, ciphertext):

- *Defines decryption function.*

d, n = private_key

- *Separates d and n from private key.*

*plain = [chr((char ** d) % n) for char in ciphertext]*

- *Converts encrypted numbers back to characters.*

return ''.join(plain)

- *Joins characters into complete string.*

7. Main Program

p = int(input("Enter first prime number: "))

q = int(input("Enter second prime number: "))

- *Takes two prime numbers from user.*

public_key, private_key = generate_keypair(p, q)

- *Generates RSA keys.*

print("\nPublic Key:", public_key)

print("Private Key:", private_key)

- *Displays generated keys.*

message = input("\nEnter message: ")

- *Takes message from user.*

encrypted_msg = encrypt(public_key, message)

- *Encrypts the message.*

print("\nEncrypted Message:", encrypted_msg)

- *Displays encrypted message.*

decrypted_msg = decrypt(private_key, encrypted_msg)

- *Decrypts encrypted message.*

print("Decrypted Message:", decrypted_msg)

- *Displays original decrypted message.*

Task 2: Create a Digital Signature:

Code:

```python
# Import required libraries
from hashlib import sha256
from Crypto.PublicKey import RSA
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
# Step 1: Generate RSA key pair
key = RSA.generate(2048)
# Store private and public keys
private_key = key
public_key = key.publickey()
print("RSA Key Pair Generated\n")
# Step 2: Original message
message = b"Hello Haroon"
print("Original Message:", message.decode())
# Step 3: Create SHA256 hash of message
hash_object = SHA256.new(message)
print("SHA256 Hash:", hash_object.hexdigest())
# Step 4: Sign the hash using private key
signature = pkcs1_15.new(private_key).sign(hash_object)
print("\nDigital Signature Created")
# Step 5: Verify signature using public key
```

```python
# Step 5: Verify signature using public key
try:
    # Verify original message hash with signature
    pkcs1_15.new(public_key).verify(hash_object, signature)
    print("Signature Verified Successfully")
except (ValueError, TypeError):
    print("Verification Failed")
# Step 6: Modify message slightly
modified_message = b"Hello haroon"
print("\nModified Message:", modified_message.decode())
# Create new hash for modified message
modified_hash = SHA256.new(modified_message)
# Step 7: Verify again using old signature
try:
    # Verification should fail because message changed
    pkcs1_15.new(public_key).verify(modified_hash, signature)
    print("Signature Verified")
except (ValueError, TypeError):
    print("Verification Failed (Message Changed)")
```

Output:

```
"C:\Users\DELL\PycharmProjects\IS assig\.venv\Scripts\python.exe" "C:\Users\DELL\PycharmProjects\IS assig\digital signature.py"
RSA Key Pair Generated

Original Message: Hello Haroon
SHA256 Hash: 847ea4e0872af231e064e2e2fda0cc6800cdcad87e2854419b197fa2a295e0b8

Digital Signature Created
Signature Verified Successfully

Modified Message: Hello haroon
Verification Failed (Message Changed)

Process finished with exit code 0
```

Line-by-Line Explanation of Task 2:

1. Import Libraries

from Crypto.PublicKey import RSA

- *Imports RSA module for key generation.*

from Crypto.Signature import pkcs1_15

- *Imports PKCS#1 v1.5 signature scheme.*

from Crypto.Hash import SHA256

- *Imports SHA256 hashing algorithm.*

2. Generate RSA Key Pair

key = RSA.generate(2048)

- *Generates 2048-bit RSA key pair.*

private_key = key

public_key = key.publickey()

- *Stores private and public keys.*

print("RSA Key Pair Generated\n")

- *Displays confirmation message.*

3. Take Message Input

message = input("Enter message: ").encode()

- *Takes message from user.*
- *Converts message into bytes format.*

print("\nOriginal Message:", message.decode())

- *Displays original message.*

4. Create SHA256 Hash

hash_object = SHA256.new(message)

- *Creates SHA256 hash of message.*

print("SHA256 Hash:", hash_object.hexdigest())

- *Displays hash value in hexadecimal format.*

5. Create Digital Signature

signature = pkcs1_15.new(private_key).sign(hash_object)

- *Signs hash using private key.*

print("\nDigital Signature Created")

- *Displays signature creation message.*

6. Verify Signature

try:

- *Starts exception handling block.*

pkcs1_15.new(public_key).verify(hash_object, signature)

- *Verifies signature using public key.*

print("Signature Verified Successfully")

- *Displays verification success message.*

except (ValueError, TypeError):

- *Handles verification errors.*

print("Verification Failed")

- *Displays failure message if signature is invalid.*

7. Modify Message

modified_message = input("\nEnter modified message: ").encode()

- *Takes modified message from user.*

modified_hash = SHA256.new(modified_message)

- *Creates hash of modified message.*

8. Verify Again

try:

- *Starts verification block again.*

pkcs1_15.new(public_key).verify(modified_hash, signature)

- *Verifies modified message with old signature.*

print("Signature Verified")

- *Displays success if verification passes.*

except (ValueError, TypeError):

- *Handles verification error.*

print("Verification Failed (Message Changed)")

- *Displays failure because message was modified.*